

Evaluating Model Context Protocol (MCP)-Enabled Large Language Models for Natural Language User Interfaces to Big Data Systems

Ruoxi Rosa Yan (1272152)
yanrr@student.unimelb.edu.au
Word count: 6061

Supervisor: Professor Richard Sinnott

Submitted September 2025, in partial fulfillment of
the conditions for the award of the degree MSc Computer Science.

School of Computing and Information Systems
THE UNIVERSITY OF MELBOURNE

Contents

1	Introduction	1
2	Background	2
2.1	LLM Agents and Tool calling	2
2.2	Model Context Protocol	4
2.3	Australian Internet Observatory	5
3	Literature Review	8
3.1	Natural Language Interface: A History	8
3.2	Evaluation of LLM agents	10
3.3	Benchmarking MCP agents	11
3.4	Conclusion	12
4	Research Questions	15
5	Methodology	16
5.1	Overview	16
5.2	Benchmark Design	16
5.3	Model Selection	19
5.4	Evaluation Metrics	19
5.5	Prototype Interface	21
6	Conclusion and contribution	24

List of Figures

1	Typical MCP workflow. Source: [1]	5
2	Screenshots illustrating current AIO design challenges.	7
3	Mock-up of the prototype chat-style interface.	22

List of Tables

1	Categories of benchmark tasks	17
2	Example user study questions and evaluation dimensions.	23

1 Introduction

In the last decade, social media platforms have become valuable data sources for studying human behaviour and informing the design of computational tools, policy interventions, and sociotechnical systems. Despite the growing availability of large-scale social media datasets, researchers often face significant barriers. Accessing these datasets often requires navigating heterogeneous APIs and writing complex queries, which requires technical expertise. As a result, researchers often spending substantial time on repetitive engineering work instead of focusing on social science analysis[2].

The Australian Internet Observatory (AIO)[3] addresses some of these challenges by aggregating large-scale social media data across multiple platforms and providing access through a uniform API and a data dashboard. While this reduces some of the engineering burden, the complexity of the API remains an obstacle for researchers without advanced technical expertise, and data exploration continues to favour technically skilled research teams, thereby limiting accessibility for scholars without programming backgrounds.

Recent advances in natural language processing, particularly large language models (LLMs), open opportunities to lower these barriers. For example, natural language-to-SQL methods have enabled the construction of natural language interfaces (NLIs) that allow users to issue queries in plain language. While they show potential for making complex data systems more accessible, such approaches are typically tailored to relational databases and do not generalise to heterogeneous, multi-platform environments such as AIO.

The Model Context Protocol (MCP) [1], introduced by Anthropic in late 2024, provides a standardized framework for connecting LLM agents to external data sources and tools. MCP offers structured, secure, and auditable channels for agentic AI systems to access and process real-time information, potentially enabling scalable and maintainable NLIs for big data research infrastructures.

Despite these technical advances, the effectiveness of MCP-enabled LLMs as natural language interfaces for big data driven research environments remains underexplored. Most existing benchmarks evaluate LLM-based agents in synthetic contexts, where carefully curated datasets and narrow tool schemas are used to test general-purpose question-answering ability. These studies reveal important insights into present capabilities and limitations of LLMs, yet they diverge significantly from the realities of research infrastructures like AIO, which involve heterogeneous data, complex orchestration across multiple APIs, and end users who are typically non-technical social science researchers. This gap highlights the need for systematic evaluation of MCP-enabled agents in the context of real-world research infrastructures.

The aim of this project is to benchmark and evaluate how effectively MCP-enabled large language models function as natural language interfaces for heterogeneous social media data within the AIO.

To achieve this aim, the project will adapt and extend existing evaluation frameworks for LLM tool use, focusing on challenges unique to heterogeneous, research-driven environments. Unlike prior studies that rely on synthetic datasets and general purpose question answering, this study situates evaluation within AIO’s diverse multi-platform collections. The study will assess not only accuracy and execution success, but also tool utilization, orchestration efficiency, and researcher-facing usability, generating insights into both system performance and research workflows, contributing a domain-specific benchmark that addresses real-world social science needs.

Specifically, the objectives of the study are:

1. To evaluate how effectively LLMs can translate natural language queries into accurate MCP tool calls within AIO.
2. To assess the ability of LLMs to dynamically orchestrate multiple tools, avoiding under-utilization and reliance on single datasets.
3. To adapt existing benchmarking frameworks for execution-grounded, heterogeneous data environments.
4. To provide researcher-facing insights on usability, reproducibility, and cost.

2 Background

This section provides a summary of the background knowledge this work relies on, including a brief introduction of LLM agents, the Model Context Protocol, and an overview of AIO.

2.1 LLM Agents and Tool calling

Large Language Models (LLMs) are Transformer-based deep learning models [4]. They are trained on vast quantities of text data, enabling them to understand, generate, and reason about human language. Traditional LLMs are constrained by static parametric knowledge: Their knowledge is frozen at the time of training, leading to a "knowledge cutoff" that prevents them from accessing real-time information. They also lack the capacity to interact with external systems or APIs, which limits their ability to perform tasks beyond text generation.

To overcome these fundamental limitations, the research community has increasingly focused on developing "LLM agents"—systems that augment LLMs with capabilities for autonomous planning, tool use, and dynamic interaction with external environments [5]. Unlike traditional models that simply generate a response based on a prompt, an LLM agent can decompose complex tasks into a sequence of intermediate steps. They actively engage with their environments through selecting and utilizing external tools to gather information, execute actions, and adapt their strategy based on observational feedback.

2.1.1 A Systematic View of Tool Use

A tool for an LLM is defined as an external program that the model can call through a function interface, passing inputs and receiving outputs [6]. Recent surveys systematically classify these tools into three categories based on their function [7]:

- Perception Tools: used to gather information from the environment, such as search engine APIs or weather services.
- Action Tools: used to change the external environment, such as posting to a social media platform or controlling connected devices.
- Computation Tools: used to perform calculations or logical operations beyond the LLM’s internal capacity, such as code interpreters or calculators. This paradigm allows LLMs to overcome core limitations such as the knowledge cutoff, factual hallucinations, and a lack of real-time information. By leveraging external tools, LLM agents can acquire up-to-date knowledge and perform complex computations and tasks beyond their pre-trained capabilities, and become capable of operating effectively in dynamic, data-rich environments.

2.1.2 Evolution of Tool-Calling Mechanisms

The mechanisms enabling tool calling in LLMs have evolved significantly. Early pioneering works like ReAct [8] and Toolformer [9] demonstrated that LLMs could be prompted to generate textual representations of tool calls. Through prompt engineering, these systems produced interleaved sequences of reasoning traces (i.e., "chain of thought") and tool-specific actions. For instance, the model first verbalizes a reasoning step (e.g., "I need to find the current weather in Melbourne"), then selects an appropriate tool and formulates an action (e.g., Action: search("current weather in Melbourne")), which was a textual command that required external code for parsing and execution. Then the model receives an observation from the tool, and iterates this process until the task is complete. This strategy enabled models to iteratively combine reasoning with external observations, and improved their abil-

ity to complete multi-step tasks involving dynamic information retrieval and planning. This approach falls under the category of tuning-free methods, as it does not require additional model training.

Building on these insights, the paradigm of “tool calling” shifted towards a more robust approach where the model is explicitly fine-tuned for native function calling. This tuning-based method involves training LLMs on high-quality datasets of function calls, which dramatically improves their ability to select the correct tool, pass the correct parameters, and work in multi-turn, multi-tool scenarios [10]. This native function-calling capability has been widely adopted and is a core feature of modern LLM APIs, including both commercial systems (e.g., GPT series[11], Gemini[12]) and open-source releases (e.g., Llama 3[13]). In parallel, frameworks such as LangChain[14] and LlamaIndex[15] have emerged to provide developers with standardized interfaces and abstractions for managing tool use.

Despite these advances, there are remaining challenges: Efficient coordination of multiple tools and the management of large toolsets, especially when dependencies are complex or incomplete [16]. This limitation underscores the need for a more sophisticated, standardized framework for managing the flow of information.

2.2 Model Context Protocol

While tool calling empowers LLMs to generate more reliable responses, the traditional implementation poses several significant challenges for large-scale deployment: Developing separate connectors for each API introduces redundant engineering effort and slows development, while the addition of new data sources or tools remains cumbersome, limiting scalability. The absence of standardized protocols further limits interoperability, preventing reuse across different models and leading to duplication. Moreover, In addition, inconsistent security practices in custom solutions increase the likelihood of misconfigurations and data breaches[17].

This challenge has been described as the “ $N \times M$ problem”[18]: Connecting N different LLMs to M external tools or data sources required developers to build a custom integration for each pair, which is both costly and difficult to maintain.

Model Context Protocol (MCP) seeks to address these limitations by offering a standardized framework for how LLM applications expose and consume tools, contextual data, and sampling instructions, acting like” a USB-C for AI”[1]. It provides a universal interface that addresses the $N \times M$ problem by replacing it with a more efficient $M + N$ model, in which each new LLM or tool integrates only once with the protocol.

MCP is designed around a structured client-server model with three core roles[19]:

- **Hosts:** Represents LLM applications, such as integrated development environments (IDEs) or platforms like Claude Desktop, which initiate communication and manages the user experience
- **Clients:** Maintains one-to-one connections within host applications to facilitate requests and responses. Instantiated by host applications.
- **Servers:** Exposes specific capabilities to AI applications, including Tools (definition of executable functions that AI applications can invoke), Resources (Data sources), and Prompts (Reusable templates that help structure interactions).

In a typical workflow (see Figure 1), the server first exposes available tools to the client as context. When the host LLM receives a natural language query, the client sends a function call to the server, and the server returns the result. This architecture enables a standardised and auditable channel for tool use across heterogeneous environments.

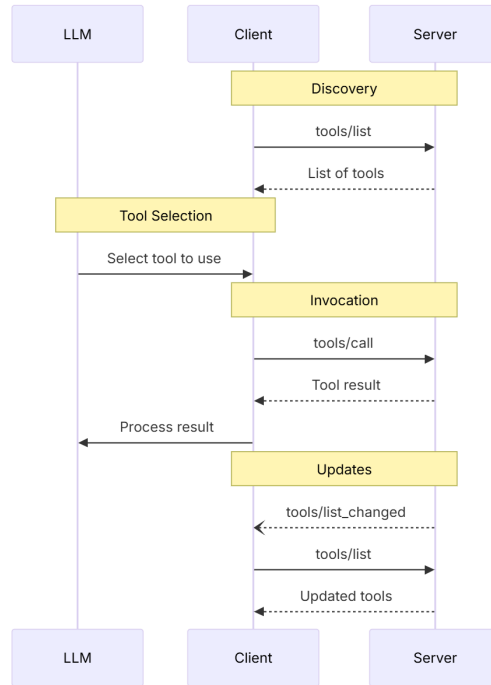


Figure 1: Typical MCP workflow. Source: [1]

2.3 Australian Internet Observatory

The Australian Internet Observatory (AIO) is a national-scale, multi-institutional digital observatory project that aggregates and analyzes data from online platforms to support research on internet activity, social media ecosystems, and digital policy. Its goal is to

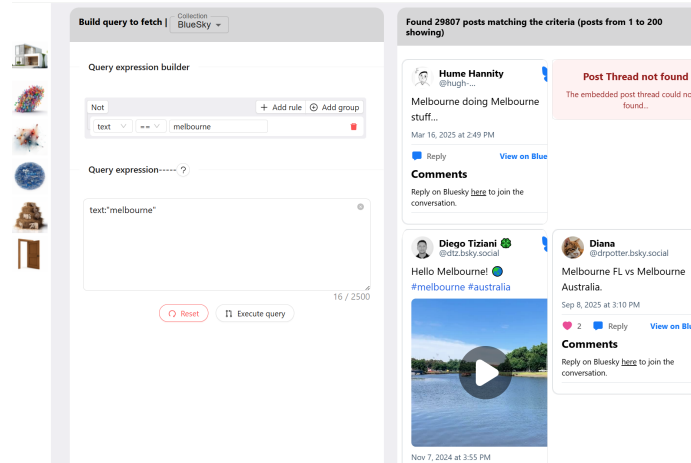
provide access to large-scale, ethically governed social media datasets and analytical tools for researchers across disciplines[20].

AIO curates and maintains collections of posts from platforms including Mastodon, Reddit, BlueSky, YouTube, and Flickr, as well as a historical Twitter corpus. To enable researchers to interact with these datasets, the Melbourne eResearch Group has developed the AIREd (Australian Internet Research Data) dashboard and API services[3]. These resources allow retrieval of post identifiers, which can then be resolved to full post content via official platform APIs. It also provides NLP tools such as topic modelling, sentiment analysis and a range of visualisation tools.

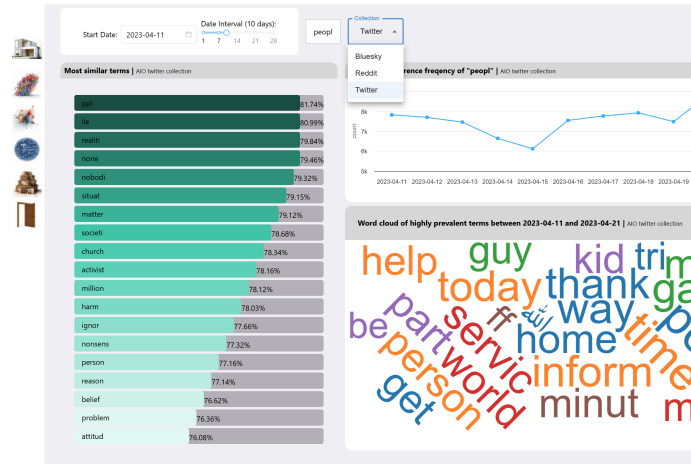
While this has significantly improved availability of social media data for research, several design challenges remain:

- **Complex Query Construction:** The query builder (2a) requires users to compose structured expressions (e.g., `text:"melbourne"`), select operators, and manage nested rules, which requires technical literacy in query syntax
- **Extensive API Documentation:** The API exposes powerful analytic functions, but interacting with them requires detailed knowledge of parameters (eg. date formats, thresholds), authentication protocols, and post-processing steps. These demands can exclude researchers without technical training.
- **Fragmentation of analytic outputs:** Visualisations such as topic graphs, aggregation plots, and word clouds are siloed across different modules, each with separate menus and parameter settings (2b, 2c). To compare results from different platforms, researchers must repeatedly configure settings or write API calls, breaking exploratory flow.

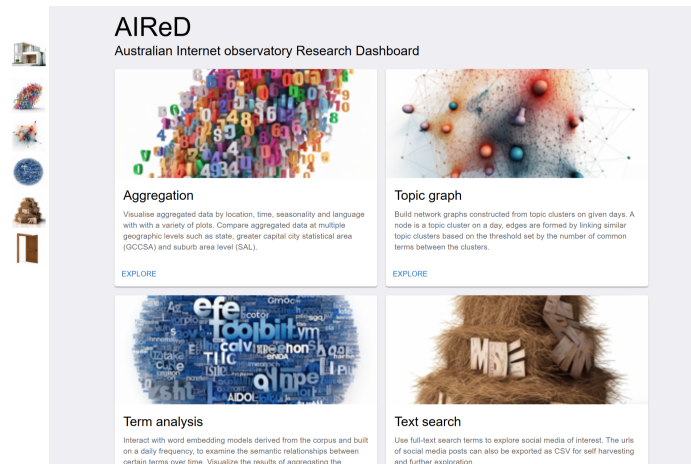
The challenges highlight the limitations of current design in technical complexity, which restrict broader engagement with AIO datasets. MCP enabled agents offer a potential solution by providing a single standardised interface to all tools. Instead of navigating menus or writing queries, researchers could simply ask: “Compare sentiment around digital privacy on Reddit and Mastodon in July 2025”, leaving the LLM agent to orchestrate the necessary tool calls in the background. This integration could substantially lower technical barriers and make AIO resources more accessible to a wider community of researchers.



(a) Query builder interface requiring structured expressions and nested rules.



(b) Hard to compare results from different platforms.



(c) Analytic outputs siloed across modules (topic graphs, aggregations, word clouds).

Figure 2: Screenshots illustrating current AIO design challenges.

3 Literature Review

3.1 Natural Language Interface: A History

Natural language interfaces (NLI) are a type of user interface that allow users to interact with a system in a conversational manner using natural language rather than formal query syntax. It has been an active area of research for decades, driven by the goal of empowering non-technical users to access and query complex datasets. Over time, the development of NLIs can be traced through several stages: from database-focused text-to-SQL, through semi-structured data, toward LLM-mediated APIs and tools. Across these stages, a consistent set of challenges emerges—faithfulness, reproducibility, and robustness—which directly motivate the current study.

A large body of work of NLI is rooted in text-to-SQL systems[21], which translates natural language queries into executable SQL statements, allowing users to interact directly with databases without requiring expertise in SQL syntax. Large-scale benchmarks such as Spider[22], CoSQL[23], and BIRD[24] highlight persistent challenges, particularly in schema generalisation, multi-turn interactions, and domain adaptation. Despite the advances brought by neural models and now LLM-based approaches, two problems remain unresolved: ensuring that generated SQL faithfully captures user intent (query fidelity), and achieving robustness when models encounter queries or schemas from previously unseen domains, where they must generalise beyond the training distributions.[21]. These problems highlight the difficulty of aligning flexible human language with rigid, structured backends.

To extend NLIs to more diverse data environments, researchers developed systems [25] for NoSQL and graph databases. While SQL-focused systems struggled with schema generalisation, these systems face challenges in query expressivity and alignment with heterogeneous backends. Systems such as Text-to-NoSQL[26] and Text2Cypher [27] demonstrate that natural language can be mapped into heterogeneous query languages, while multi-agent frameworks like NAT-NL2GQL[28] explore schema alignment and iterative error correction. However, these approaches remain constrained to single-query translation and are often limited to one backend or domain. The harder problems of retrieving data across heterogeneous sources at scale remains largely unsolved.

Parallel efforts applied NLIs to data visualisation, where the goal was to translate vague user queries into executable code for tools such as Vega-Lite[29] or matplotlib[30]. Tools like NL4DV[31] and LIDA[32] reduced usability barriers for non-technical users but also revealed problems of inconsistent query-to-visualisation mappings and poor reproducibility across different models. These findings mirror issues already documented in text-to-SQL

systems, suggesting that reproducibility is not domain-specific but a fundamental challenge across NLI research.

Recent advances in NLIs have shifted the focus from query translation toward orchestration, where LLMs coordinate multiple tools to execute complex tasks. LLMs can now act as mediators between users and APIs, selecting and invoking tools dynamically to execute open-ended tasks. The introduction of the Model Context Protocol (MCP) formalises this development by standardising how LLMs discover, invoke, and manage tools and resources.

Two recent case studies illustrate MCP’s potential across very different domains. In Attrach et al.’s study [33], a conversational system for clinical data repositories encapsulated endpoints as MCP tools and achieved an accuracy of 94% when expert evaluators compared outputs against ground truth. This demonstrated the opportunity for simplified access to sensitive structured data, while also underscoring risks such as hallucinated outputs and potential information leakage if orchestration was not tightly controlled.

A contrasting example comes from the humanities, where researchers implemented an MCP server[34] for the DraCor [35] drama corpus, enabling LLMs to autonomously query plays, corpora, and metadata. Evaluation criteria included quantitative matrices like tool correctness, tool calling efficiency, and tool use reliability, revealing both strengths and weaknesses: while LLMs often selected appropriate tools and completed tasks, results varied across repetitions, and models frequently encountered maximum sequence length constraints when processing large files.

These studies demonstrate the potential of MCP-enabled LLMs serving as domain-specific NLIs, from high-stakes medicine to exploratory research in humanities. Yet both highlight the same open problems that are central to this project: retrieval and integration across datasets, and efficient, reliable tool orchestration under real-world constraints. These challenges persist regardless of domain, suggesting they are systemic to MCP, not context-specific.

Across this trajectory of NLI studies, progress has been made in translating queries and lowering usability barriers, but the same challenges persist: schema mismatch, query faithfulness, reproducibility, and orchestration. Existing studies demonstrate feasibility but none of these studies address heterogeneous, large-scale social media environments like AIO, where researchers require reliable cross-platform retrieval and dynamic tool orchestration. This gap—evaluating MCP-enabled LLMs as NLIs for heterogeneous social media data—provides the central motivation for the present study.

3.2 Evaluation of LLM agents

The rapid evolution of large language models (LLMs) from static text generators into autonomous agents introduces new challenges for evaluation. Traditional NLP systems are assessed on static benchmarks for tasks such as text classification, summarization, or question answering. Unlike traditional systems, LLM-based agents operate in dynamic environments where they must plan, act, and adapt. As such, there is a need for holistic, process-aware methods and metrics that can capture interactive, tool-using, stateful behavior in dynamic environments[36].

Function calling emerged as a major area of agent evaluation. It is a multi step process, including intent recognition, function selection, parameter-value-pair mapping, execution, and response integration [36]. Early work has focused on one-step interactions, evaluating agents’ ability to generate the right function calls from a set of functions in an API on synthetic datasets[37]. Benchmarks such as ToolBench [38] and the Berkeley Function Calling Leaderboard (BFCL)[39] use structural accuracy metrics, such as pass rates and abstract syntax tree matching, to evaluate performance. However these matrices capture only syntactic correctness and overlook the complexity of real-world scenarios of implicit parameters, multi-turn dependencies, and long-context processing.

Recognising these limitations, successive benchmarks have moved toward richer and more realistic settings. Later versions of BFCL [39] incorporated organisational tools and multi-turn, multi-step evaluation logic, while tau -Bench[40] introduced a framework where LLMs simulate both users and environments to test tool-augmented agents in multi-turn tasks. It also proposed the Pass@k metric, designed to measure reliability and consistency across repeated trials, which is more relevant for production systems than single-shot accuracy. Results revealed that even state-of-the-art models exhibited low consistency (e.g., Pass@8 below 25% on domain-specific tasks), highlighting the gap between laboratory accuracy and production reliability[40]. While initially designed for customer service agents, variants such as MedAgentBench[41] have been extended to medical settings, showing that the methodology is adaptable across domains like healthcare.

Taken together, these efforts trace a clear trajectory: from static, single-step synthetic benchmarks toward multi-step, multi-turn, and domain-sensitive evaluation frameworks. At the same time, they expose persistent blind spots. Most current benchmarks remain confined to narrow domains or synthetic environments, without engaging the heterogeneous and dynamic APIs that characterize real-world big data infrastructures. Addressing these blind spots requires benchmarks that capture not only correctness of tool calls but also reliability, efficiency, and reproducibility in complex, evolving tool ecosystems[37].

3.3 Benchmarking MCP agents

3.3.1 Why MCP-Specific Benchmarks Matter

While these general benchmarks represent progress, they remain insufficient for evaluating MCP-enabled systems. Unlike prior studies, which often assume static tool schemas in synthetic settings, MCP agents must operate across large, heterogeneous, and evolving tool spaces, where APIs may change over time, introduce rate limits, or expose unfamiliar parameter conventions.

This introduces unique benchmarking challenges: first, MCP agents must retrieve and integrate across diverse servers, which goes beyond single-domain reasoning. Second, it requires tool orchestration at scale, requiring agents to discover, select, and sequence unfamiliar tools in long-horizon workflows. Finally, in MCP environments there are trade-offs between accuracy, latency, cost, and reproducibility, which are rarely measured in earlier synthetic or domain-specific settings.

3.3.2 Taxonomy of Existing MCP Benchmarks

Recent work[42, 43, 44, 45, 46, 47, 48, 49] has begun to explore this space, proposing a series of MCP-specific benchmarks that seek to address these challenges. Broadly, these contributions can be grouped into automation-oriented frameworks, execution-grounded evaluations, ecosystem-scale orchestration benchmarks, and workflow or behavioral analyses.

Automation-oriented frameworks: MCPEval[45] introduced an automated system for generating synthetic tasks and reference trajectories based on available tools. The evaluation process is based on evaluating the tool name, parameters, and call order against a reference standard by an LLM judge. While scalable, its reliance on synthetic tasks limits real word representativeness. MCP-RADAR[47] complemented this by introducing multi-dimensional metrics (accuracy, efficiency, resilience), providing granular diagnostics. These works demonstrate how evaluation can be scaled and systematised, but remain limited in capturing real-world complexity. Moreover, reliance on LLM-as-judge does not always reflect true execution success, as a tool call may look “correct” but fail when run[50].

Execution-Grounded Benchmarks: Other benchmarks, such as MCP-Universe[49] and LiveMCPBench [43], emphasize real MCP servers and execution-based scoring. Instead of judging responses by text similarity or LLM-judged trajectories, these frameworks check actual task outputs against ground truth, capturing real-world fragility and time-sensitivity. Results reveal that even frontier models achieve less than 50% success, with failures concentrated in long-horizon planning and tool retrieval. LiveMCP-101 further stress-tests agents with distractor tools and curated difficulty tiers. An innovative dual-agent evaluation is

introduced: one agent executes a reference plan, while the other attempts autonomous execution. Error analysis reveals that semantic errors and overconfident self-solving is the dominant failure mode. Collectively, execution-grounded approaches reveal the fragility of MCP agents in realistic, dynamic environments while providing more objective evaluation.

Ecosystem-Scale Orchestration: As MCP ecosystems expand to thousands of servers, recent benchmarks target breadth, complexity, and cross-domain orchestration. MCP-Bench emphasises multi-step, ambiguous, and cross-server tasks. Its hybrid evaluation combines rule-based checks with LLM-as-judge rubrics, enhanced with prompt shuffling to reduce bias. MCPToolBench++ [48] expand coverage to 1,509 tasks across 87 tools and multiple languages. Similarly, it employs automatic task generation and LLM-as-Judge evaluation, and introduces "abstract syntax tree directed acyclic graph (ASTDAG)" metrics for multi-interaction tasks. Both studies converge on a major bottleneck: semantic grounding, or the ability of agents to map user queries into correct and contextually valid parameters. While models often generate syntactically valid tool calls, they struggle to infer parameters that are semantically correct and contextually appropriate. These frameworks highlight the need for benchmarks that move beyond isolated tool calls toward evaluating full multi-step orchestration, dependency resolution, and cross-domain workflows, aligning closely with real-world use cases.

Workflow and behavioral analyses: Finally, some studies shift attention from task outcomes to workflow usability. For example, MCPGauge[46] evaluates short dialogues between LLMs and MCP tools to probe whether MCP actually helps models. The findings are counterintuitive: MCP tools degrade LLM performance by an average of 9.5% while greatly increasing token overhead[46]. The study also reveals models show low proactivity in one-shot queries but show significant improvement in two-turn dialogues. This has direct implications for user workflows in environments like AIO: agents may fail to use MCP when expected, researchers may face higher compute costs, and one-shot queries may go unanswered or be mishandled. This lens underscores that evaluation must also account for researcher (user) experience and efficiency, not only task accuracy.

3.4 Conclusion

3.4.1 Insights and Limitations

Despite the range of evaluation metrics proposed, existing benchmarks primarily test the general orchestration ability of LLMs rather than their suitability for heterogeneous, research-driven, data-rich environments such as those found in big data infrastructures. Several

limitations are evident. First, current benchmarks emphasize synthetic complexity, coding skills, everyday utility, or general reasoning ability, and less about data retrieval and analytics that mirror research practice. Secondly, while some domain-specific benchmarks exist, they do not address the needs of researchers working with large-scale heterogeneous datasets, such as those curated by AIO. Finally, user-facing challenges such as proactivity, compliance, and overhead (e.g., token cost, latency) are underexplored, yet central to adoption in research workflows. Moreover, many benchmarks adapt existing datasets rather than constructing tasks grounded in MCP-native challenges, introducing an evaluation bias toward general LLM capabilities. This limitation highlights the need for benchmarks grounded in social science research tasks and big data infrastructures, ensuring that MCP-enabled LLM evaluation reflects the realities of scholarly inquiry.

A recurring observation across the studies is tool under-utilization — models failing to dynamically and appropriately leverage the full set of tools available to them. LiveMCP-Bench points out that once models discover and adopt a single tool, they tend to rely exclusively on that tool, even when tasks could be better solved by orchestrating multiple tools[43]. This narrow reliance underscores a key weakness in current agents’ capacity for dynamic multi-tool reasoning. LiveMCP-101 echoes this, noting that open-source models often “under-explore tool affordances”[42] by clustering in the low-token, low-tool region. These agents stop early, avoiding broader exploration of available tools, which results in both inefficiency and missed opportunities for problem-solving. MCPGauge adds another perspective: in one-turn settings, models frequently try to answer from their own pretrained knowledge rather than calling MCP tools (e.g. GPT-4 achieving a tool invocation accuracy (TIA) of only 0.10. [46]). Case studies report similar patterns. In DraCor MCP’s deployment, models frequently hit maximum context length despite the provision of helper tools for batched data retrieval. Instead of leveraging these tools, agents attempted to process full datasets internally and failed. The authors suggest “Docstring Engineering”[34] — re-designing tool descriptions to better communicate affordances—as one potential mitigation. This points to the role of interface design in shaping utilisation.

Relatedly, context management remains an unsolved problem: without efficient discovery strategies, agents may become overwhelmed, ignore tools listed later in long contexts, or succumb to “context rot”[51], where performance degrades with extended context windows. Despite repeated observations, under-utilization is treated as a secondary finding in current benchmarks rather than a central evaluation axis.

3.4.2 Implications for AIO and this study

Despite repeated observations of tool underutilization, current studies treat it as a side finding rather than a central object of study. Few explicitly measure the extent to which tools are ignored or over-relied upon, and fewer still investigate how interface design choices (e.g., tool descriptions, context ordering) might mitigate or exacerbate the problem.

Within the AIO context, this represents a critical gap. Researcher queries often require cross-platform orchestration (e.g., aggregating sentiment on Reddit and comparing it with trends on Mastodooon). Underutilization in this setting would mean agents default to a single dataset or attempt to answer without querying data at all, producing incomplete analyses. This project therefore positions tool utilization as a core evaluation axis. By adapting insights from previous studies, the benchmark design will explicitly test whether MCP agents can dynamically leverage multiple tools in realistic research workflows. Furthermore, by incorporating considerations of tool definition design and context management, the study will examine how interface engineering interacts with agent behavior. In doing so, it extends existing MCP benchmarking literature toward a researcher-facing, usability-driven evaluation tailored to heterogeneous social media data.

4 Research Questions

RQ1: Applicability

To what extent do existing MCP benchmarking methods apply to heterogeneous big-data systems, such as the Australian Internet Observatory (AIO)?

RQ2: Performance

How accurately and reliably can LLMs translate natural language queries into accurate and faithful MCP calls across AIO’s multi-platform datasets?

RQ3: Usability

What challenges do non-technical researchers face when using LLM-driven MCP interfaces, and how can benchmarking capture these challenges in terms of tool utilization, reliability, and efficiency, as well as comparative usability against traditional workflows?

In addressing RQ3, the study will consider two key dimensions of usability:

- **Efficiency, satisfaction, and perceived accessibility:** How a chatbot-style interface for natural language querying may influence researchers’ experience.
- **Comparative outcomes:** Whether MCP interfaces can deliver richer and more accurate data insights than those obtained through standard use of the AIO dashboard.

5 Methodology

5.1 Overview

This study employs a dual methodological strategy that integrates technical benchmarking with exploratory usability evaluation, inspired by previous studies [32, 33]. To answer RQ1-2, the study designs a domain-specific benchmark tailored to AIO’s big data social media datasets. This choice responds to a key limitation in existing MCP benchmarks, which often rely on synthetic tasks or general tool schemas that fail to capture the scale and variability of real-world research environments. Evaluating LLMs under this benchmark allows systematic analysis of both their technical performance and their ability to faithfully translate natural-language queries into tool calls.

To address RQ3, complementing the benchmarking study, the project will also develop a lightweight prototype interface. This prototype functions as an experimental instrument, enabling comparative usability studies against the AIO dashboard.

This combined approach separates system-level evaluation (benchmarking the fidelity and robustness of LLM-MCP interactions) from user-level evaluation (assessing whether such a system is usable for non-technical researchers). Current MCP benchmarks mainly rely on synthetic tasks and cannot test scalability to real-world multi-platform datasets such as AIO. Conversely, a usability study without technical benchmarking risks being anecdotal and ungrounded. The dual strategy therefore provides a complementary evidence base.

5.2 Benchmark Design

The project will design a benchmark task suite tailored to AIO’s MCP capabilities. Unlike prior synthetic benchmarks, these tasks will be grounded in realistic research questions that social scientists might pose when using AIO.

Each task will be specified in three forms:

- **Natural language query:** e.g., “Compare sentiment trends on Reddit and Mastodon about housing across Australia during 2024.”
- **Reference execution plan:** a “gold standard” set of MCP tool calls (tools, parameters).
- **Expected outcomes:** single numeric values, distributions, or qualitative assertions.

The benchmark will include 30–50 tasks, divided into the categories shown in Table 1.

Table 1: Categories of benchmark tasks

Category	Description	Example
Metadata & Collection Tasks	List available datasets, return collection metadata (counts, harvest dates). Simple tasks involving one tool with straightforward parameters.	What datasets are available in AIO, and how many posts do they contain?
Single-Platform Analytical Tasks	Aggregations, keyword frequencies, sentiment distributions, time-series counts. One platform, requires selecting the correct analytic function and parameters (date range, term, granularity).	Show the monthly sentiment trend of Reddit posts about housing during 2024.
Cross-Platform Comparative Tasks	Compare trends across two or more collections. Requires orchestrating multiple tool calls and combining results.	Compare sentiment about housing on Reddit and Mastodon during 2024.
Advanced NLP & Topic Tasks	Topic clustering, similarity networks, NLP-derived metrics. Higher complexity: multiple calls, larger outputs, possible pagination.	Build a topic similarity network for Reddit discussions about elections in 2023.
Visualization Tasks	Chart generation (bar charts, time-series plots, network graphs).	Generate a network graph of topic clusters for Mastodon posts in Aug 2025.

This design ensures coverage across the MCP tool space and provides a basis for measuring both **technical applicability** (RQ1) and **fine-grained evaluation of MCP agent performance** (RQ2).

5.2.1 Dataset and Task Construction

A hybrid approach will be used to create the dataset. This ensures tasks capture both the realism of AIO workflows and the comparability of established benchmarks.

AIO Research Use Cases: Tasks will be grounded in common analyses performed by identified through AIO’s documentation, published research outputs, and existing users, to cover aspects such as topic modelling, sentiment trend analysis, keyword frequency, and time-series aggregation. These represent workflows commonly performed by social scientists. A small set of initial task templates, will be curated manually, and be iteratively

refined through a controlled LLM - GPT5, with human review to make sure they are meaningful.

Adaptation of MCP Benchmarks: To ensure comparability with prior studies, question templates from existing MCP benchmarks [43, 47] (e.g., retrieval, aggregation, orchestration) will be mapped onto AIO’s toolset. This allows direct assessment of how methods generalise from synthetic to real-world data.

Synthetic Augmentation: Controlled LLM rewriting will be used to generate paraphrased or difficulty-varied versions of queries [43, 44]. Each variant will be reviewed and validated by experts to ensure semantic equivalence and task validity. In practice, a single task may have multiple natural-language phrasings that correspond to the same execution plan, but differ in how much technical knowledge they assume. For example, one phrasing might explicitly reference a specific tool (“use the sentiment aggregation function”), while another expresses the same intent in broader terms (“show sentiment trends”), reflecting differences in researcher expertise.

5.2.2 Ground Truth Collection

Each task will be paired with a reference execution plan and expected outcomes:

Reference Execution Plan. A “gold standard” sequence of MCP tool calls will be created by manually completing the task via the API. Where multiple valid execution paths exist, the benchmark will record all acceptable variants to avoid penalising semantically correct alternatives.

Expected Outcomes. Expected outcomes will be collected via manual AIO API executions. Depending on the task type, outcomes may include structured metadata (e.g., collection IDs, harvest dates), numeric aggregations or time-series, ranked keyword lists, clusters with top terms, or graph structures for similarity networks. For visualisation tasks, only the structured data (e.g., adjacency lists, coordinate arrays) will be stored.

Model outputs will be compared against these references using task-specific criteria, such as:

- **Exact matching** for identifiers.
- **Numeric equality or tolerance thresholds** for counts.
- **Jaccard overlap** for clusters.

For time-sensitive queries (e.g., “trend until September 2025”), reference scripts and MCP executions will be run against timestamped snapshots of the AIO dataset.

5.3 Model Selection

This study will evaluate a representative sample of large language models to capture variation across both proprietary and open-weight systems. The selection strategy and deployment criteria build on prior work in this research group [52]:

Model availability and accessibility: Proprietary models (e.g., GPT-5, Claude 3.5) will be accessed through API-based deployments, while open-weight models (e.g., LLaMA-3-70B, Qwen-2-72B) will be considered where infrastructure resources permit.

Model capability: Preference will be given to models at the current state of the art, which are widely adopted in research and have demonstrated strong performance across general benchmarks.

Comparative coverage: Including both frontier proprietary systems and open-weight alternatives allows benchmarking to address two complementary perspectives: the performance of cutting-edge models accessible via commercial APIs, and the feasibility of using open models within academic infrastructure.

To simplify deployment and ensure comparability, the project will use OpenRouter[53], a unified API routing platform that provides access to a wide range of models through a single interface. This enables systematic benchmarking without the need for separate credentials or infrastructure for each provider. OpenRouter will primarily support access to proprietary systems, while open-weight models may be deployed subject to available compute resources (e.g. via NeCTAR[54]), using quantization or inference optimizations where necessary. At minimum, two proprietary and one open-weight model will be guaranteed, ensuring comparative coverage.

5.4 Evaluation Metrics

To answer RQ1 and RQ2, the benchmark will incorporate both **technical performance metrics** and **researcher-facing usability metrics** (RQ3: reliability, efficiency, and cost-effectiveness). This ensures evaluation captures not only whether MCP-enabled LLMs can complete tasks correctly, but also whether they do so in ways that are practical for researchers operating in big data environments.

5.4.1 Technical Performance Metrics (RQ2)

Task Completion Rate (TCR): Measures the proportion of tasks for which the model produces a correct outcome, scored as a binary pass/fail under task-specific criteria.

Formally, each task i is assigned a binary score $\text{pass}_i \in \{0, 1\}$ depending on whether the output matches the reference outcome under task-specific criteria (exact match, tolerance threshold, or overlap-based evaluation). Overall performance is computed as:

$$\text{TCR} = \frac{1}{N} \sum_i \text{pass}_i.$$

Since the AIO MCP server is still under development, this design should be regarded as a preliminary framework. Specific criteria will be refined once the final server implementation and data characteristics are confirmed.

Tool Utilization Metrics.

- **Tool Selection Accuracy:** Proportion of invoked tools that are valid given the available toolset. $R_{\text{valid}} = \frac{|\{e \in E: \text{tool}(e) \in T_{\text{available}}\}|}{|E|}$ [44].
- **Tool Coverage:** Proportion of required tools that were invoked, based on gold-standard execution plans. $\text{Coverage} = \frac{|\text{Tools}_{\text{used}} \cap \text{Tools}_{\text{required}}|}{|\text{Tools}_{\text{required}}|}$ for tasks with gold tool chains [42].
- **Orchestration Depth:** Maximum and average number of sequential tool calls in successful task completions, reported as a distribution. Captures the model’s ability to perform multi-step reasoning rather than one-shot calls [44].
- **Under-utilization Ratio:** Captures the extent to which agents skip tools necessary for solving a task.

Each task is first manually executed to obtain a gold execution plan, represented as one or more valid tool chains $\{\mathcal{M}_1, \mathcal{M}_2, \dots, \mathcal{M}_k\}$, where each method $\mathcal{M}_j$ is a set of tools sufficient to solve the task. Given the set of tools actually used by the agent $\mathcal{C}$, the *utilization ratio* is:

$$\text{Util} = \max_j \frac{|\mathcal{C} \cap \mathcal{M}_j|}{|\mathcal{M}_j|}.$$

The *Under-utilization Ratio* is then:

$$U = 1 - \text{Util}.$$

This metric measures how many of the gold-standard tool opportunities were left unexploited, while giving credit for completing any valid method among multiple alternative workflows. Inspired by LiveMCP-101[42]’s use of gold execution plans for process-based evaluation and MCPGauge’s emphasis on measuring “proactivity” and missed tool use.

Researcher-Facing Usability Metrics (RQ3)

Reliability

- **Pass@k Accuracy:** Success rate when the agent is allowed up to k independent attempts, reflecting robustness under repeated use [40].

Efficiency

- **Latency:** Average wall-clock time per task.
- **Average Token Consumption:** Mean number of output tokens generated per task, reflecting cost-effectiveness [47].
- **Tokens per Successful Task:** Ratio of total tokens used (input + output) to the number of successful tasks:

$$\frac{\text{Input Tokens} + \text{Output Tokens}}{\#\text{Successes}}$$

This reflects effective sample efficiency, complementing raw token totals with an efficiency-adjusted measure [49].

5.5 Prototype Interface

The interface will adopt a chat-style design, enabling researchers to submit plain-language queries and retrieve targeted social media posts. Internally, the interface will pass queries to the top performed benchmarked LLMs, which will translate them into MCP tool calls, execute against the AIREd API, and return results in structured or visualized form. The prototype serves two purposes: first, as a demonstration tool to ground benchmarking results in realistic researcher workflows; and second, as a testbed for usability evaluation. Figure 3 shows a mock up of the interface design.

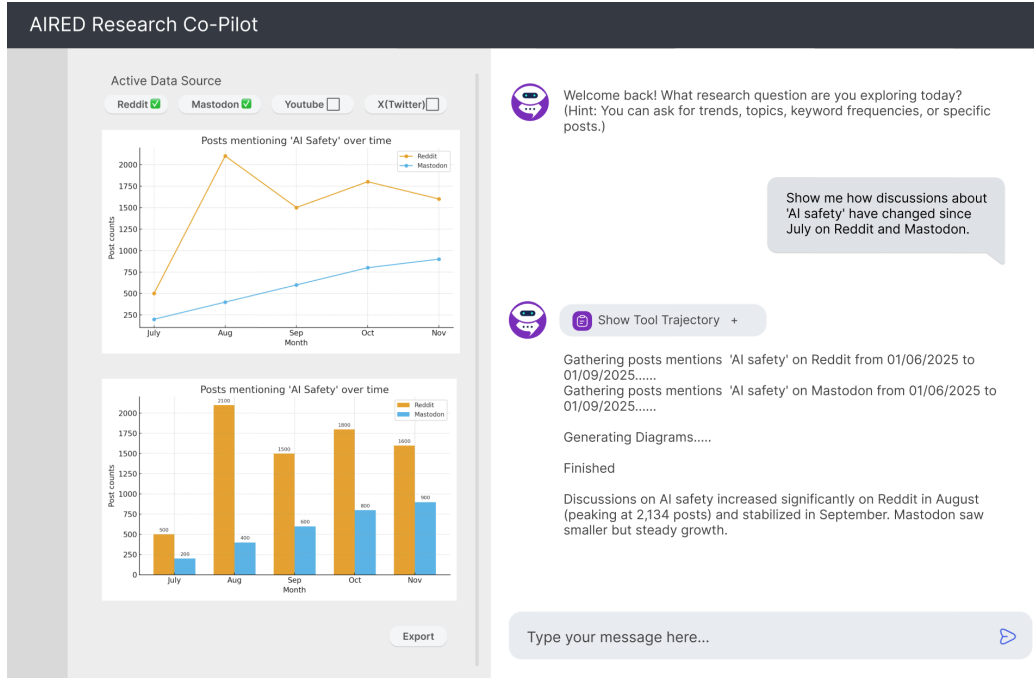


Figure 3: Mock-up of the prototype chat-style interface.

5.5.1 User Study

To address the question of whether MCP interfaces can deliver richer results than standard dashboard workflows, we will explore the prototype interface through conversational task scenarios.

An exploratory user study with 3–5 AIO-affiliated researchers will be conducted:

1. Participants receive a short orientation to both the AIO dashboard and the MCP prototype.
2. Each performs a small set of representative tasks (e.g., one single-platform, one cross-platform, one exploratory).
3. Tasks are counterbalanced: half start with the dashboard, half with the MCP interface, to avoid order bias.

Comparative analysis will consider both quantitative measures (e.g., time taken, number of tasks completed, variety of analytical dimensions covered) and qualitative feedback (e.g., user perceptions of efficiency, accessibility, and interpretability). Table 2 shows example user study questions.

Table 2: Example user study questions and evaluation dimensions.

Dimension	Example Question/Prompt	Type of Measure
Task Efficiency	“Complete the following query: Show sentiment trends for Reddit posts about housing in 2024.”	Time taken, errors, completion success
Cross-Platform Orchestration	“Compare Reddit and Mastodon discussions of climate change over the last six months.”	Tasks completed, variety of tools used
Exploration & Richness	“What additional insights about housing affordability can you retrieve from the data?”	Number of analytical dimensions covered (counts, distributions, examples)
Interpretability	“How clear and useful were the outputs you received?”	Likert-scale rating (1–5), open-ended feedback
Accessibility & Usability	“How easy was it to formulate this query using the chat interface compared to the dashboard?”	Likert-scale rating (1–5), qualitative comments
Overall Preference	“Which interface would you prefer for your research tasks, and why?”	Open-ended feedback

6 Conclusion and contribution

This project investigates the effectiveness of Model Context Protocol (MCP)-enabled large language models as natural language interfaces (NLIs) for heterogeneous, big-data research infrastructures. Using the Australian Internet Observatory (AIO) as a case study, it designs and implements a domain-specific benchmark that reflects realistic research workflows, moving beyond synthetic tasks common in current evaluation frameworks. The study integrates both technical benchmarking and exploratory usability testing, assessing accuracy, tool utilization, orchestration depth, and researcher-facing dimensions such as efficiency and accessibility. By situating evaluation within AIO’s diverse multi-platform collections, the project aims to capture challenges unique to research-driven environments and advance understanding of how LLM agents can support social science inquiry.

This work makes three main contributions:

1. **Domain-Specific Benchmark:** Development of a benchmark suite grounded in AIO workflows, capturing cross-platform orchestration, complex query construction, and researcher-oriented outputs.
2. **Evaluation Framework Extension:** Introduction of metrics such as the under-utilization ratio and orchestration depth to systematically measure tool use, complementing task accuracy with process-aware evaluation.
3. **Prototype and Usability Study:** Implementation of a chat-style MCP prototype and comparative user study, providing empirical evidence of how MCP interfaces affect accessibility, efficiency, and research outcomes relative to existing dashboards.

Together, these contributions provide both methodological advances for benchmarking MCP-enabled agents and practical insights into their applicability for big data research infrastructures.

References

- [1] [Online]. Available: <https://modelcontextprotocol.io/docs/getting-started/intro>
- [2] A. Alvarado Garcia, T. Yang, and M. Miceli, “What knowledge do we produce from social media data and how?” *Proc. ACM Hum.-Comput. Interact.*, vol. 9, no. 1, Jan. 2025. [Online]. Available: <https://doi.org/10.1145/3701216>
- [3] AIReD, “Aired website,” <https://www.aio.ereseach.unimelb.edu.au/>, 2025. [Online]. Available: <https://www.aio.ereseach.unimelb.edu.au/>
- [4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in neural information processing systems*, 2017, pp. 5998–6008.
- [5] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou, R. Zheng, X. Fan, X. Wang, L. Xiong, Y. Zhou, W. Wang, C. Jiang, Y. Zou, X. Liu, Z. Yin, S. Dou, R. Weng, W. Cheng, Q. Zhang, W. Qin, Y. Zheng, X. Qiu, X. Huang, and T. Gui, “The rise and potential of large language model based agents: A survey,” 2023. [Online]. Available: <https://arxiv.org/abs/2309.07864>
- [6] Z. Wang, Z. Cheng, H. Zhu, D. Fried, and G. Neubig, “What are tools anyway? a survey from the language model perspective,” 2024. [Online]. Available: <https://arxiv.org/abs/2403.15452>
- [7] Y. Qin, S. Hu, Y. Lin, W. Chen, N. Ding, G. Cui, Z. Zeng, Y. Huang, C. Xiao, C. Han, Y. R. Fung, Y. Su, H. Wang, C. Qian, R. Tian, K. Zhu, S. Liang, X. Shen, B. Xu, Z. Zhang, Y. Ye, B. Li, Z. Tang, J. Yi, Y. Zhu, Z. Dai, L. Yan, X. Cong, Y. Lu, W. Zhao, Y. Huang, J. Yan, X. Han, X. Sun, D. Li, J. Phang, C. Yang, T. Wu, H. Ji, Z. Liu, and M. Sun, “Tool learning with foundation models,” 2024. [Online]. Available: <https://arxiv.org/abs/2304.08354>
- [8] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” 2023. [Online]. Available: <https://arxiv.org/abs/2210.03629>
- [9] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” 2023. [Online]. Available: <https://arxiv.org/abs/2302.04761>
- [10] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, L. Hong, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun,

- “Toollm: Facilitating large language models to master 16000+ real-world apis,” 2023. [Online]. Available: <https://arxiv.org/abs/2307.16789>
- [11] OpenAI, “Gpt-4 technical report,” 2023.
- [12] G. Team, R. Anil, S. Borgeaud, Y. Wu, J.-B. Alayrac, *et al.*, “Gemini: A family of highly capable multimodal models,” 2023.
- [13] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, “Llama: Open and efficient foundation language models,” 2023.
- [14] H. Chase, “Langchain,” <https://github.com/langchain-ai/langchain>, 2022.
- [15] J. Liu and L. Team, “Llamaindex: A data framework for llm applications,” 2024.
- [16] C. Qu, S. Dai, X. Wei, H. Cai, S. Wang, D. Yin, J. Xu, and J.-r. Wen, “Tool learning with large language models: a survey,” *Frontiers of Computer Science*, vol. 19, no. 8, Jan. 2025. [Online]. Available: <http://dx.doi.org/10.1007/s11704-024-40678-2>
- [17] A. Singh, A. Ehtesham, S. Kumar, and T. T. Khoei, “A survey of the model context protocol (mcp): Standardizing context to enhance large language models (llms),” *Preprints*, April 2025. [Online]. Available: <https://doi.org/10.20944/preprints202504.0245.v1>
- [18] M. Treiber, “Model context protocol (mcp): A new standard for tool and data integration,” Apr 2025. [Online]. Available: <https://www.ikangai.com/model-context-protocol-mcp-a-new-standard-for-tool-and-data-integration/>
- [19] Anthropic, “Architecture overview.” [Online]. Available: <https://modelcontextprotocol.io/docs/learn/architecture>
- [20] “Australian Internet Observatory — internetobservatory.org.au,” <https://internetobservatory.org.au/>, [Accessed 21-09-2025].
- [21] W. Zhang, Y. Wang, Y. Song, V. J. Wei, Y. Tian, Y. Qi, J. H. Chan, R. C.-W. Wong, and H. Yang, “Natural language interfaces for tabular data querying and visualization: A survey,” 2024. [Online]. Available: <https://arxiv.org/abs/2310.17894>
- [22] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Wang, and D. Radev, “Spider: A large-scale human-labeled dataset for complex and cross-domain text-to-SQL,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Brussels, Belgium: Association

- for Computational Linguistics, oct-nov 2018, pp. 3911–3921. [Online]. Available: <https://aclanthology.org/D18-1425>
- [23] T. Yu, R. Zhang, H. Er, S. Li, E. Xue, B. Pang, X. V. Lin, Y. Tan, T.-J. Shi, Z. Li, Y. Jiang, M. Yasunaga, S.-Y. Shim, T. Chen, A. Fabbri, Z. Li, L. Chen, Y. Zhang, S. Dixit, V. Zhang, C. Xiong, D. Radev, R. Socher, and W.-t. Xiong, “CoSQL: A conversational text-to-SQL challenge towards cross-domain natural language interfaces to databases,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Hong Kong, China: Association for Computational Linguistics, nov 2019, pp. 3013–3024. [Online]. Available: <https://aclanthology.org/D19-1293>
- [24] J. Li, B. Hui, G. Qu, B. Li, J. Yang, B. Li, L. Zong, H. Kang, Y. Lu, N. Huo, Y. Piao, Y. Qian, Y. He, W. Zhang, J.-G. Lou, Y.-L. Yang, and F.-Y. Wang, “BIRD: Can large language models truly understand user intentions? a case study on text-to-SQL,” 2023.
- [25] M. Liu and J. Xu, “Nli4db: A systematic review of natural language interfaces for databases,” 2025. [Online]. Available: <https://arxiv.org/abs/2503.02435>
- [26] J. Lu, Y. Song, Z. Qin, H. Zhang, C. Zhang, and R. C.-W. Wong, “Bridging the gap: Enabling natural language queries for nosql databases through text-to-nosql translation,” 2025. [Online]. Available: <https://arxiv.org/abs/2502.11201>
- [27] M. G. Ozsoy, L. Messallem, J. Besga, and G. Minneci, “Text2cypher: Bridging natural language and graph databases,” 2024. [Online]. Available: <https://arxiv.org/abs/2412.10064>
- [28] Y. Liang, T. Xie, G. Peng, Z. Huang, Y. Lan, and W. Qian, “Nat-nl2gql: A novel multi-agent framework for translating natural language to graph query language,” 2024. [Online]. Available: <https://arxiv.org/abs/2412.10434>
- [29] A. Satyanarayan, D. Moritz, K. Wongsuphasawat, and J. Heer, “Vega-lite: A grammar of interactive graphics,” *IEEE Transactions on Visualization & Computer Graphics (Proc. InfoVis)*, 2017. [Online]. Available: <http://idl.cs.washington.edu/papers/vega-lite>
- [30] J. D. Hunter, “Matplotlib: A 2d graphics environment,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.
- [31] A. Narechania, A. Srinivasan, and J. Stasko, “Nl4dv: A toolkit for generating analytic specifications for data visualization from natural language queries,” *IEEE*

- Transactions on Visualization and Computer Graphics*, vol. 27, no. 2, p. 369–379, Feb. 2021. [Online]. Available: <http://dx.doi.org/10.1109/TVCG.2020.3030378>
- [32] V. Dibia, “Lida: A tool for automatic generation of grammar-agnostic visualizations and infographics using large language models,” 2023. [Online]. Available: <https://arxiv.org/abs/2303.02927>
- [33] R. A. Attrach, P. Moreira, R. Fani, R. Umeton, and L. A. Celi, “Conversational llms simplify secure clinical data access, understanding, and analysis,” 2025. [Online]. Available: <https://arxiv.org/abs/2507.01053>
- [34] P. Trilcke, I. Börner, H. Sluyter-Gäthje, D. Skorinkin, F. Fischer, and C. Milling, “Agentic dracor and the art of docstring engineering: Evaluating mcp-empowered llm usage of the dracor api,” 2025. [Online]. Available: <https://arxiv.org/abs/2508.13774>
- [35] F. Fischer, I. Börner, M. Göbel, A. Hechtel, C. Kittel, C. Milling, and P. Trilcke, “Programmable Corpora: Introducing DraCor, an Infrastructure for the Research on European Drama,” in *Proceedings of DH2019: "Complexities", Utrecht, July 9–12, 2019*. Utrecht University, 2019.
- [36] A. Yehudai, L. Eden, A. Li, G. Uziel, Y. Zhao, R. Bar-Haim, A. Cohan, and M. Shmueli-Scheuer, “Survey on evaluation of llm-based agents,” 2025. [Online]. Available: <https://arxiv.org/abs/2503.16416>
- [37] M. Mohammadi, Y. Li, J. Lo, and W. Yip, “Evaluation and benchmarking of llm agents: A survey,” in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*, ser. KDD ’25. ACM, Aug. 2025, p. 6129–6139. [Online]. Available: <http://dx.doi.org/10.1145/3711896.3736570>
- [38] Q. Xu, F. Hong, B. Li, C. Hu, Z. Chen, and J. Zhang, “On the tool manipulation capability of open-source large language models,” 2023.
- [39] S. G. Patil, H. Mao, C. Cheng-Jie Ji, F. Yan, V. Suresh, I. Stoica, and J. E. Gonzalez, “The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models,” in *Forty-second International Conference on Machine Learning*, 2025.
- [40] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, “ τ -bench: A benchmark for tool-agent-user interaction in real-world domains,” 2024. [Online]. Available: <https://arxiv.org/abs/2406.12045>

- [41] Y. Jiang, K. C. Black, G. Geng, D. Park, J. Zou, A. Y. Ng, and J. H. Chen, “Medagentbench: A realistic virtual ehr environment to benchmark medical llm agents,” 2025. [Online]. Available: <https://arxiv.org/abs/2501.14654>
- [42] M. Yin, D. Shen, S. Xu, J. Han, S. Dong, M. Zhang, Y. Hu, S. Liu, S. Ma, S. Wang, S. R. Indurthi, X. Wang, Y. Chen, and K. Song, “Livemcp-101: Stress testing and diagnosing mcp-enabled agents on challenging queries,” 2025. [Online]. Available: <https://arxiv.org/abs/2508.15760>
- [43] G. Mo, W. Zhong, J. Chen, X. Chen, Y. Lu, H. Lin, B. He, X. Han, and L. Sun, “Livemcpbench: Can agents navigate an ocean of mcp tools?” 2025. [Online]. Available: <https://arxiv.org/abs/2508.01780>
- [44] Z. Wang, Q. Chang, H. Patel, S. Biju, C.-E. Wu, Q. Liu, A. Ding, A. Rezazadeh, A. Shah, Y. Bao, and E. Siow, “Mcp-bench: Benchmarking tool-using llm agents with complex real-world tasks via mcp servers,” 2025. [Online]. Available: <https://arxiv.org/abs/2508.20453>
- [45] Z. Liu, J. Qiu, S. Wang, J. Zhang, Z. Liu, R. Ram, H. Chen, W. Yao, S. Heinecke, S. Savarese, H. Wang, and C. Xiong, “Mcpeval: Automatic mcp-based deep evaluation for ai agent models,” 2025. [Online]. Available: <https://arxiv.org/abs/2507.12806>
- [46] W. Song, H. Zhong, Z. Ding, J. Xue, and Y. Li, “Help or hurdle? rethinking model context protocol-augmented large language models,” 2025. [Online]. Available: <https://arxiv.org/abs/2508.12566>
- [47] X. Gao, S. Xie, J. Zhai, S. Ma, and C. Shen, “Mcp-radar: A multi-dimensional benchmark for evaluating tool use capabilities in large language models,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.16700>
- [48] S. Fan, X. Ding, L. Zhang, and L. Mo, “Mcptoolbench++: A large scale ai agent model context protocol mcp tool use benchmark,” 2025. [Online]. Available: <https://arxiv.org/abs/2508.07575>
- [49] Z. Luo, Z. Shen, W. Yang, Z. Zhao, P. Jwalapuram, A. Saha, D. Sahoo, S. Savarese, C. Xiong, and J. Li, “Mcp-universe: Benchmarking large language models with real-world model context protocol servers,” 2025. [Online]. Available: <https://arxiv.org/abs/2508.14704>
- [50] J. He, J. Shi, T. Y. Zhuo, C. Treude, J. Sun, Z. Xing, X. Du, and D. Lo, “From code to courtroom: Llms as the new software judges,” 2025. [Online]. Available: <https://arxiv.org/abs/2503.02246>

- [51] K. Hong, A. Troynikov, and J. Huber, “Context rot: How increasing input tokens impacts llm performance,” Chroma, Tech. Rep., July 2025. [Online]. Available: <https://research.trychroma.com/context-rot>
- [52] L. Morandini, S. Winter, and R. O. Sinnott, “Correctness comparison of open-weight and proprietary llms for spatial tasks,” in *2024 IEEE International Conference on Future Machine Learning and Data Science (FMLDS)*, 2024, pp. 91–95.
- [53] “OpenRouter — openrouter.ai,” <https://openrouter.ai/>, [Accessed 21-09-2025].
- [54] “ARDC Nectar Research Cloud — Australian Research Data Commons — ardc.edu.au,” <https://ardc.edu.au/services/ardc-nectar-research-cloud/>, [Accessed 21-09-2025].